IILM UNIVERSITY

School of Computer Science and Engineering

Python-Based Network Packet Sniffer

Internship Project Presentation

Student Name: Riha Singh

Enrollment Number: CS-2341156

Program : B.Tech CSE

Semester: 7

Organization Name: IILM University Greater Noida

Project Overview

- A Python-based network packet sniffing application developed as an internship project.
- Uses Scapy to capture and inspect network packets on a user-selected interface.
- Provides interactive controls for interface selection, packet count, and protocol filtering.
- Uses the Linux ip link command to display and validate available network interfaces.
- Captured packets are displayed with their protocol-layer details for analysis.

Problem Statement

The Challenge

Raw network traffic contains large volumes of packet information. Manually identifying relevant packets and inspecting their fields can be difficult for learners and analysts.

Project Need

A lightweight tool is needed to select an interface, control the number of packets, optionally filter by protocol, and immediately inspect captured traffic.

Approach

Automate packet capture and inspection through Python and Scapy while using native Linux networking commands for interface discovery and validation.

Objectives

- Capture live network packets from a selected network interface.
- Display available interfaces before starting a capture session.
- Validate the interface entered by the user before sniffing begins.
- Allow the user to specify the number of packets to capture.
- Support optional protocol-based BPF filtering.
- Display detailed packet contents using Scapy's packet inspection functionality.
- Provide a simple repeat/exit workflow for multiple capture sessions.

Technologies and Tools Used

Python

Core programming language used to implement the interactive packet-sniffing workflow.

Scapy

Python networking library used for packet capture and detailed packet-layer inspection.

Linux Networking

The ip link command is used to enumerate and validate network interfaces.

subprocess

Python module used to execute ip link and collect its output programmatically.

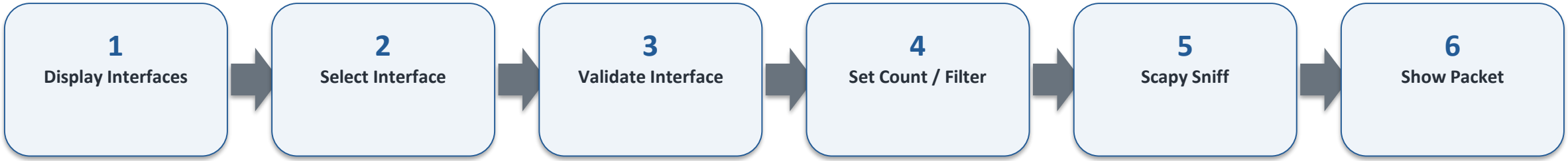
BPF Filter

Protocol filters entered by the user are passed to Scapy's sniff function.

Terminal

Provides the interactive interface for selecting options and viewing packet details.

System Architecture



Network traffic → selected interface → optional BPF protocol filter → Scapy packet capture → show_packet(packet) → packet details

Methodology / Working Process

- 1. Interface discovery: subprocess.run() executes ip link and extracts interface names.
- 2. Interface validation: is_valid_interface() checks whether the selected interface exists.
- 3. User configuration: packet limit and protocol-filter choice are collected.
- 4. Capture setup: sniff() receives iface, prn, count, and optional filter parameters.
- 5. Packet processing: each captured packet is passed to show_packet().
- 6. Inspection: packet.show() displays the packet's available layers and fields.
- 7. Session control: the user can start another capture or exit.

Key Implementation Components

Packet Capture

`sniff(iface=interface, prn=show_packet, count=..., filter=...)` starts the capture using the selected configuration.

Packet Inspection

`show_packet(packet)` calls `packet.show()` to print detailed packet information.

Interface Discovery

`subprocess.run(['ip','link'], capture_output=True, text=True, check=True)` obtains Linux interface information.

Validation

`is_valid_interface()` parses ip link output and confirms that the selected interface is present.

Filtering

When enabled, the user's protocol expression is passed to Scapy as the capture filter.

User Interaction and Output

Step 1 — Interface

The application displays available interfaces and asks the user to select one.

Example categories may include loopback or Ethernet/Wi-Fi interfaces, depending on the Linux system.

Step 2 — Capture Settings

The user specifies:
- Number of packets
- Whether to apply a protocol filter
- The protocol/filter expression when required

Step 3 — Packet Details

Captured packets are passed to show_packet(), which uses packet.show() to display available packet layers and fields in the terminal.

Testing and Observations

- Interface selection is checked before packet capture, reducing errors caused by invalid interface names.
- A packet count of 0 is interpreted as no capture limit by the application logic.
- Protocol filtering is optional, allowing both broad captures and focused analysis.
- Scapy provides structured access to packet layers and fields through packet.show().
- The application supports repeated capture sessions without restarting the program.
- The implementation is suitable as a learning and analysis tool for understanding live network traffic.

Learning Outcomes and Future Scope

Learning Outcomes

- Practical use of Python for networking
- Packet capture with Scapy
- Linux interface management
- Command execution through subprocess
- Protocol filtering concepts
- Packet-layer inspection
- Interactive command-line application design

Future Enhancements

- Add a graphical dashboard
- Save captures to PCAP files
- Add packet summaries and statistics
- Support richer protocol filters
- Add timestamps and structured logging
- Generate traffic-analysis reports
- Add alerting for selected traffic patterns

Conclusion & References

Conclusion

The project demonstrates a practical Python-based approach to live packet capture and inspection. By combining Scapy with Linux interface discovery and user-controlled filtering, the application provides a compact environment for learning and analyzing network traffic.

References

1. Scapy Documentation — Packet manipulation and sniffing
2. Python Documentation — subprocess module
3. Linux ip-link manual — network interface management
4. Internship project source code and implementation notes

THANK YOU